DEVOPS REAL-TIME INTERVIEW Q&A
🟢 **PART-1 — DAILY WORK + INCIDENT MANAGEMENT**
Q1 → Q20 | Zero Knowledge → Real-Time Job Level

1️⃣ DAILY WORK & TICKET HANDLING

---

1. How do you pick tickets?

🧠 **Simple Meaning**
Different problems come from developers, QA, customers, and monitoring tools.
We call these **tickets**.
Example:
Ticket 1 → Website down
Ticket 2 → Login issue for 5 users
Ticket 3 → Change server name
Ticket 4 → Production API returning 500
We cannot solve all of them at the same time.
So first, we pick the **most important issue**.

🎯 **Interview Answer**
"I prioritize tickets based on business impact, severity, number of affected users, and urgency. I usually handle production-critical issues first, followed by high-priority tasks and then normal requests."

🔥 **Real-Time Example**
Suppose 5 tickets arrive at 10 AM:
P1 → Production website down
P2 → API slow
P3 → Developer needs access
P3 → Disk cleanup
P4 → Documentation update
I handle them in this order:
P1 Website Down
↓
Investigate
↓
Restore service
↓
Monitor
↓
P2 API issue
↓
P3
↓
P4

🛠️ **Actual DevOps Work**
I open Jira/ServiceNow.
I check the ticket details:

- Priority
- Impact
- Affected service
- Environment
- Error
- Requester
- Deadline

Then I decide the priority.

⚠️ **Interview Point**
Do not say:
"I always work on tickets in the order they came."
Correct:
"I prioritize work based on impact, urgency, and priority."

2️⃣ What is P1/P2?

🧠 **Simple Meaning**
P = **Priority**
Usually:
P1 → Critical
P2 → High
P3 → Medium
P4 → Low

🔥 **P1 Example**
Production website is completely down.

Users → ❌ Website
Business is completely affected.
So:
P1

🔥 **P2 Example**
Website is working, but the payment API is slow.
Website → ✅
Payment → ⚠️ Slow
It is an important issue, but it is not a complete outage.
So:
P2

🎯 **Interview Answer**
"P1 is a critical production issue with major business or user impact, such as a complete service outage. P2 is a high-priority issue that has significant impact but does not completely stop the service."

⚠️ **Important**
Priority definitions can **change from company to company**.
In an interview:
"The exact priority definitions depend on the organization's incident policy."
This sounds professional.

3️⃣ **How many tickets do you handle daily?**

🧠 **Simple Meaning**
The interviewer is actually testing **how you manage your workload**, rather than just the number of tickets.

❌ **Bad Answer**
"I handle exactly 10 tickets every day."
Why?
In real DevOps work, one production incident itself can take 5 hours.

🎯 **Better Interview Answer**
"It depends on the complexity and priority of the tickets. On a normal day I may handle several small requests or a few major incidents. During a critical production incident, I focus primarily on restoring the service rather than ticket count."

🔥 **Real-Time**
Normal day:
3 small tickets
2 deployment tasks
1 monitoring issue
2 access requests
But a production outage occurs:
Production outage
↓
Stop normal work
↓
Incident investigation
↓
Restore service
↓
RCA

⚠️ **Interview Point**
Ticket count is less important than impact and resolution quality.

4️⃣ **What is a ticket lifecycle?**

🧠 **Simple Flow**
From ticket creation to closure:
OPEN
↓
ASSIGNED
↓
IN PROGRESS
↓
BLOCKED (if needed)
↓
RESOLVED
↓
CLOSED

🔥 **Real-Time Example**
Developer:
"Production deployment failed."
The developer creates a ticket.
OPEN
DevOps engineer accepts the ticket:

IN PROGRESS
Checks pipeline logs.
Fixes the issue.
Tests it.
RESOLVED
After the requester verifies it:
CLOSED
🎯 **Interview Answer**
"A ticket generally starts in Open state, gets assigned and moved to In Progress, may become Blocked if there is a dependency, then moves to Resolved after the fix is verified, and finally Closed after confirmation."

5️⃣ **How do you track your work?**
🧠 **Simple Meaning**
We use a tracking system so that we do not forget the work we have done.
Common tools:

- Jira
- ServiceNow
- Azure Boards
- GitHub Issues

🔥 **Real-Time**
Suppose the ticket is:
DEVOPS-2456
Production API issue
Daily update:
10:00 AM → Started investigation
11:00 AM → Found high CPU
12:00 PM → Scaled service
01:00 PM → Monitoring stable
02:00 PM → Resolved
🎯 **Interview Answer**
"I track my work through Jira or ServiceNow. I keep ticket status, investigation notes, blockers, actions taken, and resolution details updated so the team has clear visibility."
⚠️ **Professional Point**
Do not write only:
"Fixed."
In the ticket, document:
What happened?
What did I check?
What was the root cause?
What action was taken?
What is the current status?

6️⃣ **What do you do if a ticket is blocked?**
🧠 **Simple Meaning**
You want to work on the ticket, but there is a dependency on another team.
Example:
DevOps
↓
Needs DB access
↓
DBA approval required
↓
BLOCKED
🔥 **Real-Time**
Ticket:
"Deploy application to production."
But production credentials/access are unavailable.
I:
Check access
↓
Confirm missing permission
↓
Contact IAM/Cloud team
↓
Update Jira
↓
Wait for access
↓
Continue deployment

🎯 **Interview Answer**
"If a ticket is blocked, I first identify the exact dependency, communicate with the responsible team, document the blocker in the ticket, and escalate when necessary so that the work can continue."
⚠️ **Important**
Do not silently leave a blocked ticket.
Communicate + document + escalate.

7️⃣ **How do you prioritize tasks?**
🧠 **Simple Formula**
To decide priority:
Impact

- **Urgency**
- **Business importance**
- **Number of users affected**
- **SLA**
- **Dependencies**

🔥 **Example**
Task A → Production down
Task B → Developer wants a new server
Task C → Documentation
Order:
A
↓
B
↓
C
🎯 **Interview Answer**
"I prioritize tasks based on severity, business impact, urgency, affected users, SLA requirements, and dependencies. Production-critical issues always take precedence over routine tasks."

2️⃣ **INCIDENT MANAGEMENT & RCA**

8️⃣ **What is Incident Management?**
🧠 **Simple Meaning**
There is a problem in production.
The goal is:
First restore the service.
The root cause can be investigated in detail later.
🔥 **Example**
Website:
Users
↓
Load Balancer
↓
EC2
↓
Application
Suddenly:
HTTP 500
Incident management flow:
Alert
↓
Acknowledge
↓
Investigate
↓
Mitigate
↓
Restore
↓
Monitor
↓
RCA

🎯 **Interview Answer**
"Incident management is the process of detecting, responding to, mitigating, and resolving production issues while minimizing business and user impact."
⚠️ **Key Difference**
Incident Management = Restore service quickly
RCA = Understand why it happened

❓ **What is RCA?**
RCA = **Root Cause Analysis**
🧠 **Simple Meaning**
A problem occurred.
The question is:
"Why did this problem actually happen?"
🔥 **Example**
Website is down.
Immediate observation:
EC2 CPU = 100%
But CPU at 100% may not be the root cause.
Further investigation:
CPU 100%
↓
Application process
↓
Memory leak
↓
Traffic increased
↓
Application bug
Actual root cause:
Application memory leak under high traffic.
🎯 **Interview Answer**
"RCA is the process of identifying the underlying cause of an incident rather than only fixing the visible symptom. The goal is to prevent the issue from happening again."
🔥 **RCA usually includes:**
What happened?
When did it happen?
Who/what was affected?
Why did it happen?
How was it detected?
What fixed it?
How do we prevent it?

🔟 **How do you handle pressure in production?**
🧠 **Simple Meaning**
During a production outage, we should not panic.
❌ **Bad Approach**
Server is down!
Restart everything!
Delete the pod!
Change random configuration!
This can make the problem worse.
✅ **Correct Approach**
Stay calm
↓
Understand impact
↓
Check monitoring
↓
Follow runbook
↓
Mitigate
↓
Communicate
↓
Monitor
↓
RCA
🎯 **Interview Answer**
"During production incidents, I stay calm and follow the incident response process. I first understand the impact, check monitoring and logs, apply the safest mitigation, communicate regularly with stakeholders, and then perform RCA after service restoration."

⭐ **Strong Interview Line**
"During a critical incident, my first priority is service restoration, followed by root-cause analysis and preventive actions."

1️⃣ 1️⃣ **Give an example of a critical incident you handled.**
🧠 **Interviewer expects STAR style**
S → Situation
T → Task
A → Action
R → Result
🔥 **Example**
Situation:
A production API suddenly started returning HTTP 500 errors.
Task:
My responsibility was to restore the service quickly.
Action:
CloudWatch
↓
Checked CPU/memory
↓
Checked ALB target health
↓
Checked application logs
↓
Found unhealthy instances
↓
Removed unhealthy targets
↓
Scaled healthy instances
Result:
Traffic recovered and the error rate returned to normal.
🎯 **Interview Answer**
"In one production incident, the application started returning 500 errors. I checked CloudWatch metrics, ALB target health, and application logs. I identified unhealthy instances, removed them from serving traffic, scaled healthy capacity, and monitored the service until the error rate returned to normal. Afterward, I documented the RCA and preventive actions."
⚠️ **Only claim incidents you can actually explain technically.**

1️⃣ 2️⃣ **What tools do you use to monitor incidents?**
🧠 **Main Categories**

Type	Tools
Metrics	Prometheus, CloudWatch
Dashboards	Grafana
Logs	ELK, CloudWatch Logs
Kubernetes	kubectl
Alerts	Alertmanager, CloudWatch
APM/Tracing	OpenTelemetry, etc.

🔥 **Kubernetes Example**
kubectl get pods
Then:
kubectl logs <pod>
Then:
kubectl describe pod <pod>
🎯 **Interview Answer**
"I use CloudWatch for AWS monitoring and Prometheus with Grafana for Kubernetes and application metrics. For incidents, I correlate metrics, logs, events, and traces to identify the problem."

1️⃣ 3️⃣ **How do you tell stakeholders about incidents?**
🧠 **Simple Meaning**
We should not explain a technical problem to business people using too much technical jargon.
❌ **Bad**
"Pod eviction occurred because of node pressure."
The stakeholder may not understand it.
✅ **Better**
"The application is currently experiencing elevated errors. We identified the affected infrastructure and are working on restoring normal service."
🔥 **Incident Update Structure**
What happened?
↓

Impact?
↓
What are we doing?
↓
Current status?
↓
Next update?

🎯 **Interview Answer**

"I communicate the impact, current status, actions being taken, and expected next update in clear language. I avoid unnecessary technical details and keep stakeholders updated throughout the incident."

1️⃣4️⃣ **How do you prevent incidents in production?**

🧠 **Simple Meaning**

Fixing a problem after it happens is one thing.
Preventing the problem from **happening again** is more important.

🔥 **Prevention Methods**

Monitoring

- Automated testing
- Code review
- Infrastructure as Code
- Health checks
- Deployment strategy
- Backups
- Capacity planning
- Post-mortems

Example

Repeated disk-full issue.
First time:
Clean disk
But prevention:
Disk monitoring
↓
80% warning
↓
90% critical
↓
Alert
↓
Log rotation

🎯 **Interview Answer**

"I prevent incidents through proper monitoring, health checks, testing, controlled deployments, capacity planning, automation, and learning from previous incidents through RCA and post-mortems."

1️⃣5️⃣ **What is an On-Call Rotation?**

🧠 **Simple Meaning**

Production issues can happen 24/7.
Everyone cannot stay awake and work 24/7.
So team members take turns handling production support.
Example:
Monday → Ravi
Tuesday → Suresh
Wednesday → Arun
Thursday → Priya

🔥 **Incident**

2 AM:
Production alert
↓
On-call engineer
↓
Acknowledges alert

↓
Investigates
🎯 **Interview Answer**
"On-call rotation is a scheduled support model where engineers take turns being responsible for production incidents outside normal working hours. It ensures continuous operational coverage."
⚠️ **Important**
Being an on-call engineer does **not** mean you have to solve every problem alone.
You can involve the required teams.

1️⃣6️⃣ **Alert vs Incident?**
🧠 **Very Important Interview Question**
Alert
The monitoring system says:
"Something may be wrong."
Example:
CPU > 80%
Alert.
But the application may still be working perfectly.
Incident
There is actual user/business impact:
Website unavailable
Incident.
Flow
Metric abnormal
↓
ALERT
↓
Engineer investigates
↓
Actual service impact?
↓
YES
↓
INCIDENT
🎯 **Interview Answer**
"An alert is a notification generated by a monitoring system when a defined condition is detected. An incident is a confirmed event that requires response because it impacts or threatens service availability, performance, or users."

🔥 **INTERVIEW QUICK REVISION — Q1–Q16**
Ticket
↓
Prioritize
↓
P1/P2
↓
Work on ticket
↓
Incident occurs
↓
Detect
↓
Acknowledge
↓
Investigate
↓
Mitigate
↓
Restore
↓
Monitor
↓
RCA
↓
Prevent recurrence

3️⃣ **GIT & JENKINS**

1️⃣7️⃣ **What is Git used for?**
🧠 **Simple Meaning**
Git = **A version control system used to track code changes.**

Developer changes code:
v1
↓
v2
↓
v3
Git maintains the history.

🔥 **Example**

A developer accidentally breaks the production code.
Git has the previous stable commit.
git log
We can identify the previous commit.

🎯 **Interview Answer**

"Git is a distributed version control system used to track source-code changes, collaborate across developers, manage branches, and revert to previous versions when required."

Common Commands

git clone
git status
git add
git commit
git push
git pull
git branch
git merge
git log

1️⃣8️⃣ **What is a branch in Git?**

🧠 **Simple Meaning**

Main production code:
main
New feature needs a separate branch:
main
|
└── **feature/login**
The developer works on the feature branch.
The main code remains safe.

🔥 **Example**

git checkout -b feature/payment
Work complete:
feature/payment
↓
Pull Request
↓
Code Review
↓
Merge
↓
main

🎯 **Interview Answer**

"A Git branch is an independent line of development used to work on features, fixes, or releases without directly affecting the main branch."

1️⃣9️⃣ **How do you fix merge conflicts?**

🧠 **Simple Meaning**

Two developers changed the same lines in the same file.
Git gets confused:
Developer A → Version A
Developer B → Version B
Git asks:
"Which one should I keep?"

🔥 **Example**

git pull origin main
Conflict:
<<<<<<< HEAD

Developer A code

=======

Developer B code

>>>>>>> main
We manually decide which code is correct.
Then:
git add .
git commit
git push

🎯 **Interview Answer**

"I first identify the conflicting files, review both changes, resolve the conflict based on the intended functionality, run tests, and then commit and push the resolved changes."

⚠️ **Important**

After resolving a conflict, **do not push without testing.**

2️⃣0️⃣ **What is Jenkins used for?**

🧠 **Simple Meaning**

Jenkins = **Automation server**

Manual process:

Developer writes code
↓
Build manually
↓
Test manually
↓
Create artifact
↓
Deploy manually

Jenkins automates this process:

Git
↓
Jenkins
↓
Build
↓
Test
↓
Artifact
↓
Deploy
↓
Monitor

🔥 **Real-Time Example**

Developer:
git push
Webhook sends a signal to Jenkins.
Jenkins:
Checkout code
↓
Build
↓
Unit tests
↓
Docker image build
↓
Push image
↓
Deploy to Kubernetes
↓
Health check

🎯 **Interview Answer**

"Jenkins is a CI/CD automation server. We use it to automate source-code checkout, build, testing, artifact creation, and deployment processes."

⭐ **Strong Interview Answer**

"In a typical pipeline, a developer pushes code to Git, Jenkins is triggered through a webhook, the pipeline builds and tests the application, creates an immutable artifact such as a Docker image, and deploys it to the target environment after required validations or approvals."

🔥 **Q17–Q20 REAL-TIME FLOW**

Remember this one diagram:

👨‍💻 **Developer**
|

| git push
↓
🟦 **Git Repository**
|
| webhook
↓
🟧 **Jenkins**
|
├── Checkout
|
├── Build
|
├── Test
|
├── Security Scan
|
├── Docker Build
|
├── Push Image
|
↓
🟩 **Deployment**
|
↓
☸️ **Kubernetes / EC2**
|
↓
🌍 **Users**

If the interviewer asks:

"Explain your DevOps workflow."

You can say:

"Developers commit and push code to Git. Jenkins is triggered through a webhook. The pipeline checks out the code, builds the application, runs tests and security checks, creates the deployment artifact, and deploys it to the target environment. After deployment, we verify health using monitoring, logs, and application checks. If a critical issue occurs, we roll back to the last stable version."